STAGE 2

Pet Model Training & Deployment

Theory • Implementation • Evaluation • API • Docker • Interview Preparation

Project: Oxford-IIIT Pet breed classification

Model: ImageNet-pretrained ResNet-18 with partial fine-tuning

Stack: Python, PyTorch, Torchvision, CUDA, FastAPI, Docker concepts

Purpose: Beginner-friendly theory to understand why each step exists and prepare to discuss the project in interviews.

Project results to remember

Measure	Recorded result
Classes	37 pet breeds
Train / validation / test	5,141 / 1,096 / 1,112 images
Best validation accuracy	90.33%
Held-out test accuracy	89.57% (996 / 1,112 correct)
Test loss	0.3728
Hardware	NVIDIA RTX 3050 Laptop GPU, 6 GB VRAM

Study principle: be able to trace a new image through preprocessing, the trained model, and the API response—and explain the reasoning behind each choice.

How to use this guide

Read the sections in order the first time. Later, revise the question bank aloud and verify each concept against your code. Example snippets explain concepts; they are not a substitute for checking your actual implementation.

Contents

- 1. Stage 2 overview and pipeline
- 2. Transfer learning and ResNet-18
- 3. Fine-tuning and trainable layers
- 4. GPU, CUDA, tensors, batches, and VRAM
- 5. Training loop, loss, gradients, and optimizers
- 6. Validation, checkpoints, and data leakage
- 7. Evaluation metrics and results
- 8. Inference and checkpoint loading
- 9. Reproducibility and model packaging
- 10. FastAPI and REST APIs
- 11. Validation, HTTP errors, and API tests
- 12. Docker and containers
- 13. Reliability, performance, and security
- 14. Limitations and future improvements
- 15. Interview-ready explanation
- 16. Interview questions and model answers
- 17. Revision sheet and glossary

Scope note

This guide uses the recorded Stage 2 project setup and results. Confirm exact filenames, API response keys, Docker status, and command-line instructions against your current repository before quoting them as implemented features.

1. Stage 2 overview and pipeline

Stage 2 advances the project from a custom CNN baseline to a pretrained ResNet-18, fine-tunes it for 37 pet breeds, evaluates it on held-out data, and serves predictions through an inference API.

End-to-end pipeline

```
Images + breed labels
       |
       v
Dataset / DataLoader
       |
       v
Resize + tensor conversion + normalization
       |
       v
ImageNet-pretrained ResNet-18
       |
       +-- frozen early layers
       +-- trainable layer4
       +-- new trainable 37-class classifier
       |
       v
37 raw logits per image
       |
       +-- training: loss -> gradients -> parameter updates
       +-- inference: scores -> predicted breed
       |
       v
Best checkpoint -> test evaluation
       |
       v
FastAPI endpoint -> client response
```

Baseline versus Stage 2

Dimension	Custom CNN baseline	Stage 2
Initialization	Random weights	ImageNet-pretrained weights
Feature learning	Learns features from project data	Reuses broad visual features, then adapts selected layers
Goal	Baseline classifier	Improve accuracy and create an inference path
Engineering	Dataset and training	Transfer learning, evaluation, API and packaging

The dataset split is 5,141 training images, 1,096 validation images, and 1,112 test images. The best validation accuracy was 90.33%. The held-out test evaluation achieved 89.57% accuracy: 996 correct predictions from 1,112 images, with test loss 0.3728.

Training updates parameters. Validation guides development and checkpoint selection. Test data estimates final performance and should not be repeatedly used to tune the model.

2. Transfer learning and ResNet-18

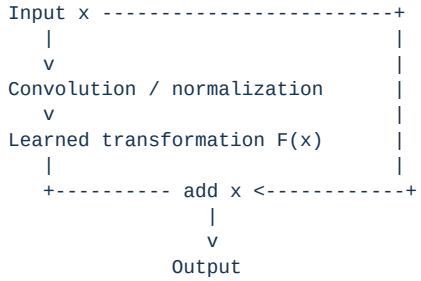
Transfer learning reuses knowledge learned on one task or dataset for another related task. ResNet-18 pretrained on ImageNet has already learned useful visual representations, such as edges, textures, shapes, and combinations of visual patterns. Starting from these weights can be more effective than learning every feature from random initialization on a modest dataset.

Why transfer learning helps

- Can reduce training time and the amount of task-specific data needed.
- Provides a useful visual representation before the model sees pet-breed examples.
- Still requires adapting the classifier and sometimes later backbone layers.
- Is not guaranteed to win: results depend on data, preprocessing, domain similarity, and training choices.

What is ResNet?

ResNet means Residual Network. A residual block learns a transformation F(x) and adds a shortcut, often the identity x: y = F(x) + x when dimensions match. If dimensions differ, the shortcut can use a projection. This helps information and gradients flow through deeper networks.



ResNet-18 at a high level

ResNet-18 contains an initial convolution and pooling operation, residual stages commonly named layer1 through layer4, global average pooling, and a final fully connected classifier. The “18” refers to the architecture's depth convention, not to the number of output classes.

Why replace the final layer?

The original classifier is designed for ImageNet's label set. This project has 37 breed classes, so the final fully connected layer must produce 37 scores. The backbone extracts features; the new head maps those features to the pet classes.

```
Image -> ResNet feature extractor -> feature vector
      -> new Linear layer with 37 outputs
      -> one logit per breed
```

Interview answer

“I chose ImageNet-pretrained ResNet-18 because it already provides useful visual features. I replaced its original classifier with a 37-class layer, froze most of the backbone, and fine-tuned the final residual stage to adapt the representation to pet breeds.”

3. Fine-tuning and trainable layers

Architecture is the network structure; parameters are the learned numerical values such as weights and biases. Fine-tuning continues training on the target task and updates selected pretrained parameters.

Term	Meaning	PyTorch control
Freeze	Do not update selected parameters	requires_grad = False
Unfreeze	Allow gradients and updates	requires_grad = True
Evaluation mode	Change behavior of BatchNorm / Dropout	model.eval()
Disable gradient tracking	Avoid building a gradient graph for inference	torch.no_grad() / inference_mode()

These are distinct controls. requires_grad affects parameter gradients; train()/eval() affects module behavior. A frozen BatchNorm layer may still update running statistics if the model remains in training mode. Decide deliberately how BatchNorm should behave during fine-tuning.

Your strategy

- Load the transfer-learning checkpoint and use a compatible 37-class model definition.
- Freeze the backbone parameters initially.
- Unfreeze layer4, the last major residual stage.
- Keep the new final classifier trainable.
- Use a smaller learning rate for pretrained layer4 than for the newly initialized head.
- Save the checkpoint that performs best on validation data, rather than assuming the final epoch is best.

Why unfreeze layer4?

Early layers often capture more general patterns such as edges and textures. Later layers combine them into higher-level representations. Training layer4 allows some task-specific adaptation while retaining much of the pretrained backbone and limiting the number of trainable parameters.

Why different learning rates?

The new classifier starts with newly initialized weights and may need larger updates. Pretrained features already contain useful information, so smaller updates can help preserve them. This is often called discriminative learning rates.

```
# Conceptual example
for p in model.parameters():
    p.requires_grad = False
for p in model.layer4.parameters():
    p.requires_grad = True
model.fc = nn.Linear(model.fc.in_features, 37)

# Optimizer: smaller LR for layer4,
# larger LR for the new classifier.
```

Feature extraction versus fine-tuning

Feature extraction freezes the pretrained backbone and trains a new head. Partial fine-tuning also updates selected pretrained layers. Transfer learning is the broader strategy that includes both approaches.

4. GPU, CUDA, tensors, batches, and VRAM

Deep learning involves large numbers of tensor operations. GPUs are designed to perform suitable operations in parallel, often accelerating training. A GPU does not automatically make every program faster: data loading, CPU operations, small workloads, and data transfers can become bottlenecks.

Term	Meaning
CPU	General-purpose processor coordinating program control and many data operations.
GPU	Parallel processor well suited to neural-network tensor operations.
CUDA	NVIDIA platform/software ecosystem for supported GPU computation.
CUDA-enabled PyTorch	PyTorch build able to dispatch supported operations to an NVIDIA GPU.
VRAM	GPU memory for weights, activations, gradients, optimizer state, and batches.
Device	Where a tensor or model resides, commonly CPU or CUDA GPU.

Tensor shape

PyTorch vision models commonly use N × C × H × W: batch, channels, height, width. Your batch shape 32 × 3 × 224 × 224 represents 32 RGB images at 224 × 224. A single image has shape 3 × 224 × 224; inference adds a batch dimension to create 1 × 3 × 224 × 224.

Why batches?

Batches process multiple examples together. Larger batches may improve hardware utilization but use more memory. Smaller batches reduce memory use but mean more updates per epoch and noisier gradient estimates. The best choice depends on model size, resolution, VRAM, and training behavior.

What consumes VRAM?

- Model weights and biases.
- Intermediate activations needed for backpropagation.
- Gradients for trainable parameters.
- Optimizer state, especially for adaptive optimizers.
- Input batches and temporary kernel workspaces.

Out-of-memory remedies

Reduce batch size or image resolution, consider mixed precision if supported and validated, avoid retaining unnecessary tensors, or use gradient accumulation. These options affect speed and sometimes numerical behavior, so re-evaluate the resulting model.

Reproducibility

Record Python, PyTorch, Torchvision, CUDA/runtime, GPU, dataset split, seeds, model configuration, and training settings. Some GPU operations can be nondeterministic; reproducibility usually means repeating the same setup and obtaining similar results, not necessarily bit-identical outputs.

5. Training loop, loss, gradients, and optimizers

Training repeatedly predicts, measures error, computes gradients, and updates trainable parameters. This cycle—not simply showing images to a model—is what makes the network learn.

One batch

```
1. Read a batch of images and labels.
2. Move tensors to the selected device.
3. Clear old gradients: optimizer.zero_grad()
4. Forward pass: logits = model(images)
5. Compute loss from logits and labels.
6. Backpropagate: loss.backward()
7. Update parameters: optimizer.step()
8. Repeat for the next batch.
```

Logits and softmax

A 37-class classifier outputs 37 raw scores called logits. Logits are not probabilities; they can be negative and need not sum to one. Softmax transforms them into nonnegative values that sum to one across classes. Softmax is useful for display, but its values are not automatically calibrated real-world probabilities.

CrossEntropyLoss

CrossEntropyLoss is standard for single-label multiclass classification. It penalizes the model when it assigns insufficient probability to the correct class. In PyTorch it expects raw logits and integer class indices; log-softmax and negative log-likelihood are handled internally. Do not normally add softmax before CrossEntropyLoss.

```
Correct class gets high probability -> lower loss
Correct class gets low probability  -> higher loss

Training:   logits + label -> CrossEntropyLoss
Inference:  logits -> softmax -> readable class scores
```

Backpropagation and gradients

Backpropagation applies the chain rule to calculate how loss changes with respect to parameters through the computational graph. A gradient describes the local change in loss associated with a parameter. The optimizer uses gradients to update trainable parameters.

Optimizer and learning rate

An optimizer defines how parameters are updated. SGD uses gradients and may use momentum. Adam maintains moving estimates of gradients and squared gradients. AdamW decouples weight decay from the gradient update. Learning rate controls update size: too high may destabilize learning; too low may make it slow.

Term	Definition
Batch	Group of examples processed together.
Iteration / step	One batch's forward/backward/update cycle.
Epoch	One complete pass through the training set.
Learning rate	Scale of parameter updates.
Weight decay	Regularization discouraging overly large weights.

Why zero gradients?

PyTorch accumulates gradients by default. optimizer.zero_grad() clears the previous batch's gradients so the next backward pass starts from the intended state.

6. Validation, checkpoints, and data leakage

Training data updates parameters. Validation data monitors development performance and supports choices such as checkpoint selection and hyperparameter tuning. Test data is reserved for a final evaluation after decisions are complete.

Split	Updates weights?	Purpose
Training	Yes	Learn parameters from labeled examples.
Validation	No	Monitor development; select checkpoints/settings.
Test	No	Estimate final performance after model selection.

Checkpoint

A checkpoint is a saved snapshot of model state, sometimes including optimizer state, epoch, and metrics. Saving the best validation checkpoint avoids assuming the final epoch is best. The recorded best validation accuracy was 90.33%.

Why validation can fluctuate

Training optimizes the training objective, not validation accuracy directly. Batches vary, the model can overfit, and decision boundaries shift. Validation performance can improve, plateau, or decrease across epochs.

Overfitting and generalization

Overfitting means learning training-specific details that do not generalize. One warning is improving training performance while validation performance stalls or deteriorates. Generalization is the ability to perform well on unseen examples.

Data leakage

Leakage occurs when information that should be unavailable during training or model selection improperly influences the model. Examples include the same image in both training and test, or repeatedly tuning decisions based on test results. Keep the test set separate from iterative experiments.

Preprocessing consistency

Training, validation, test, and inference should use compatible preprocessing. For pretrained ResNet-18, normalization should match the chosen pretrained weights' conventions. Random augmentation belongs in training; validation, test, and inference should normally be deterministic.

Fair comparisons

Comparisons are not apples-to-apples if they use different resizing, normalization, class mapping, or test examples. Keep evaluation preprocessing constant when comparing checkpoints.

7. Evaluation metrics and results

Accuracy is useful but incomplete. A model can have strong overall accuracy while failing on some classes. Per-class metrics and a confusion matrix reveal the pattern of errors.

Accuracy

Accuracy = correct predictions / total predictions. Your held-out test evaluation gave 996 / 1,112 ≈ 89.57%. This describes performance on this test set; it does not mean every prediction is 89.57% certain.

Metric	Question	Formula
Precision	When this class is predicted, how often is it right?	TP / (TP + FP)
Recall	Of true examples of this class, how many were found?	TP / (TP + FN)
F1	Balance between precision and recall.	2 × P × R / (P + R)

TP = true positives, FP = false positives, FN = false negatives. In multiclass classification these can be calculated for each class against all other classes.

Macro, weighted, and micro averages

- Macro average: each class gets equal weight; useful when each breed matters equally.
- Weighted average: class metrics are weighted by support, so larger classes contribute more.
- Micro average: aggregate counts across classes first; for single-label multiclass classification, micro precision/recall/F1 often equal accuracy.

Confusion matrix

Rows commonly represent actual classes and columns predicted classes, though plotting conventions vary. Diagonal entries are correct predictions; off-diagonal entries show confusions. For breeds, this can reveal classes that look similar in images.

Recorded test results

Measure	Result
Test images	1,112
Correct predictions	996
Test accuracy	89.57%
Test loss	0.3728
Macro F1	approximately 0.89
Weighted F1	approximately 0.90

Some difficult classes reported

Breed	F1
American Pit Bull Terrier	0.63
Staffordshire Bull Terrier	0.66
American Bulldog	0.77
Ragdoll	0.77
Birman	0.79

These metrics indicate where to investigate, not the cause of errors. Similar appearance, sample counts, pose, lighting, backgrounds, and label noise are hypotheses. Inspect actual mistakes before choosing a remedy.

8. Inference and checkpoint loading

Inference uses a trained model to make predictions without updating weights. It needs a forward pass, but no loss-driven backward pass or optimizer update.

Inference pipeline

```
Image file
 -> validate and decode
 -> convert to RGB
 -> resize / tensor / normalize
 -> add batch dimension: 1 x 3 x 224 x 224
 -> move tensor to model device
 -> model.eval()
 -> torch.inference_mode()
 -> 37 logits
 -> softmax / argmax / top-k
 -> breed name + scores
```

Why preprocessing must match

If training used RGB images normalized with ImageNet statistics but inference uses raw pixels, another channel order, or different normalization, inputs shift and predictions may degrade. Class-index-to-name mapping must be exactly consistent with training.

Control	Does	Does not
model.eval()	Switches BatchNorm / Dropout to evaluation behavior.	Disable autograd by itself.
torch.no_grad()	Disables gradient recording in a context.	Switch model to eval mode.
torch.inference_mode()	Avoids autograd overhead for inference computations.	Load weights or validate file content.

argmax and top-k

argmax returns the index of the largest score. Map it through the same class ordering used during training. Top-k returns the k highest-scoring classes and can be helpful when breeds are visually ambiguous.

Confidence caveat

Softmax normalizes scores across classes but neural networks can be overconfident. A score of 0.92 is not automatically a calibrated 92% real-world probability. Calibration should be measured if confidence drives important decisions.

Checkpoint compatibility

Recreate a compatible architecture and 37-class head before loading the state_dict. A mismatch between a 1,000-output ImageNet head and a 37-output pet head causes tensor-shape errors. Only load serialized checkpoints from trusted sources.

9. Reproducibility and model packaging

A checkpoint is not the entire application. A deployable model needs the architecture, weights, class mapping, preprocessing, dependencies, and serving code that agree with each other.

What to package or document

- Architecture definition and 37-class output dimension.
- Selected checkpoint and how it was produced.
- Class names in exact training order.
- Image conversion, resizing, tensor conversion, and normalization.
- Python and dependency versions.
- API input contract and response schema.
- Evaluation report, known limitations, and setup instructions.

PyTorch state_dict

A state_dict maps parameter and buffer names to tensors. It is commonly saved separately from the Python class defining the architecture. To reload it, construct a compatible model first and then load the saved values.

Record	Why
Dataset and split files	Keep train/validation/test membership stable.
Seed and augmentation	Help reproduce data and training choices.
Architecture / trainable layers	Define which parameters were adapted.
Loss / optimizer / LR / epochs / batch	Define optimization settings.
Software and hardware versions	Aid compatibility and reproducibility.
Checkpoint metric	Explain why the model was selected.

GitHub hygiene

Datasets may have licensing, size, and redistribution constraints; checkpoints may be large. A clean repository normally contains code, dependency instructions, configuration, README, and permitted small test assets. Document how to obtain data rather than uploading it indiscriminately. Never commit secrets or private local paths.

Model versioning

Keep the checkpoint, preprocessing, class mapping, metrics, and code version together. Otherwise, a code change can silently break compatibility or change the meaning of output indices.

10. FastAPI and REST APIs

An API is an interface that lets one program request work from another. The FastAPI service accepts an image and returns a prediction so a client does not need to import the training code directly.

What FastAPI provides

- Routes that map HTTP methods and paths to Python functions.
- Request parsing and validation using type hints and supported request types.
- Automatic OpenAPI schema generation and interactive documentation.
- JSON response serialization and standard HTTP status handling.
- An ASGI interface suitable for serving web workloads.

Common routes in this project

Route	Purpose
GET /	Root or health-style response to check that the service responds.
POST /predict	Accept an uploaded image and return a prediction.
GET /docs	Interactive Swagger UI generated by FastAPI for inspecting and testing endpoints.

HTTP methods

GET is commonly used to retrieve a resource or status. POST submits data for processing, such as an uploaded image. Status codes communicate the result separately from the JSON body.

```
Client sends POST /predict
        + image as multipart/form-data
                  |
                  v
FastAPI validates upload
 -> decode / preprocess
 -> model inference
 -> map class index to breed
 -> return JSON response
```

Why separate training from serving?

Training is an offline process that changes weights and may take minutes or hours. Serving loads a chosen checkpoint and performs inference for requests. Separating the two avoids retraining on every request and allows a stable model version to serve clients.

Response schema

```
{
  "predicted_breed": "Example Breed",
  "confidence": 0.91
}
```

This is an illustrative schema only. Use the exact response keys and score representation in your implementation and document them for callers.

11. Validation, HTTP errors, and API tests

A robust API does not assume every client sends a valid image. Validate input before preprocessing and provide useful client errors rather than unhandled server exceptions.

Validation layers

- Check that a file was supplied.
- Restrict accepted media types, but do not trust the Content-Type header alone as proof of valid image bytes.
- Decode the image and handle malformed or truncated content.
- Convert to RGB so grayscale, palette, and RGBA images have consistent channels.
- Set upload-size limits to reduce memory-exhaustion risk.
- Return clear errors for invalid inputs.

Useful HTTP status codes

Code	Meaning	Example
200	OK	Prediction succeeded.
400	Bad Request	Application-defined invalid request.
413	Content Too Large	Upload exceeds the configured limit.
415	Unsupported Media Type	Unsupported Content-Type.
422	Unprocessable Entity	Missing or invalid request field under FastAPI validation.
500	Internal Server Error	Unexpected server-side failure.

Understanding HTTP 415

415 means the server rejected the request media type. If the endpoint expects multipart/form-data, sending JSON or a raw body with an incompatible Content-Type can trigger it. In Swagger UI, choose the file field and let the browser construct the multipart request boundary; do not manually force an incompatible header.

API test checklist

Test	Expected behavior
Valid JPEG / PNG	200 response and documented prediction schema.
Missing file	Clear 4xx response; server remains healthy.
Text renamed .jpg	Rejected as invalid image content.
Unsupported media type	Clear documented 4xx response.
Oversized upload	Rejected by size limit.
Grayscale / RGBA image	Consistently converted or rejected clearly.
Checkpoint missing	Clear startup failure or unhealthy status.
Repeated requests	Stable schema and no progressive memory growth.

Testing levels

Unit tests check preprocessing and label mapping. API tests check routes and status codes. Integration tests verify that the real checkpoint loads and predicts. A smoke test checks that the deployed service starts and responds.

12. Docker and containers

Docker packages an application and its user-space dependencies into an image. A container is a running instance of that image. The purpose is to reduce environment differences and make runtime setup more repeatable.

Concept	Meaning
Dockerfile	Recipe that describes how to build an image.
Image	Built package containing app files and runtime layers.
Container	Running instance of an image.
Registry	Place to store and distribute images.
Volume	Persists or shares files outside the container's writable layer.
Port mapping	Connects a host port to a port inside the container.

Why containerize the API?

- Pin runtime dependencies and make startup repeatable.
- Run the same application package on another compatible host.
- Define a consistent working directory and startup command.
- Support deployment and CI automation.
- Separate application packaging from the host's Python installation.

Conceptual workflow

```
Dockerfile + code + dependency list
                |
                v
            docker build
                |
                v
             image
                |
                v
            docker run
                |
                v
      API inside container + port mapping
```

What belongs in the image?

Usually include application code and dependency declarations. Decide deliberately how the checkpoint is supplied: copied into the image, mounted as a volume, or fetched from a controlled artifact store. Large datasets and caches should not be included automatically.

GPU caveat

Docker does not automatically give a container access to the host NVIDIA GPU. GPU containers require compatible host drivers, container runtime/toolkit setup, and compatible framework dependencies. A CPU container is often the simplest first deployment; add GPU support only when the target environment needs it.

Common failures

- Incorrect port mapping or binding only to 127.0.0.1 inside the container.
- Checkpoint path wrong or checkpoint absent from the image/mount.
- Python or dependency version mismatch.
- Unnecessarily large image due to datasets and caches.
- Secrets or local machine paths baked into the image.

A Dockerfile is not proof of deployment

Verify that the image builds, the container starts, the health route responds, and a real prediction request works. Document the exact tested commands and runtime assumptions.

13. Reliability, performance, and security

A deployed model is software, not just a neural network. Quality includes correctness, latency, useful errors, safe input handling, dependency management, and operational visibility.

Load the model once

Model construction and checkpoint loading are relatively expensive. Normally load the model at application startup and reuse it for requests rather than rebuilding it per prediction. Check the server worker configuration: multiple workers may each load a separate model copy.

Term	Meaning
Latency	Time for one request to receive a response.
Throughput	Requests processed per unit time.
Cold start	Initial startup cost, such as imports and loading weights.
Batch inference	Processing multiple inputs together when service design permits.
Concurrency	Handling requests that overlap in time.

Common performance bottlenecks

- Large uploads or expensive image decoding.
- Repeated model initialization or checkpoint loading per request.
- CPU-only execution or excessive CPU/GPU transfer overhead.
- Too many workers loading separate model copies.
- Unbounded concurrency or memory pressure.
- Slow I/O or excessive synchronous logging.

Reliability practices

- Expose health/readiness checks and verify that weights loaded successfully.
- Log errors and timings without retaining uploaded images unnecessarily.
- Pin dependencies and test the documented install path.
- Set reasonable upload limits and timeouts.
- Keep response schemas and status codes consistent.
- Automate tests for preprocessing, class mapping, and API behavior.

Security basics

- Treat uploads as untrusted; validate content, dimensions, and size.
- Do not expose stack traces, local paths, environment variables, or secrets.
- Keep dependencies updated and avoid unnecessary privileges.
- Use authentication, rate limits, TLS, and access controls if exposed beyond trusted local use.
- Do not log or retain images without a defined purpose and appropriate policy.

Monitoring

Monitor request volume, latency, error rates, memory, model-load failures, and prediction distributions. If real-world inputs change, monitor data drift and compare sampled outcomes with verified labels when available. A service can remain online while model quality degrades.

14. Limitations and future improvements

A credible project explanation includes limitations. A single accuracy number does not explain every error or guarantee performance on all real-world images.

Current limitations to understand

- Closed-set classifier: it chooses among 37 known breeds even if the image is unknown or contains no pet.
- Similar breeds may be confused; pose, lighting, crop, background, and image quality can influence predictions.
- Softmax confidence is not automatically calibrated.
- Test performance reflects this dataset's distribution and may not transfer to every camera or domain.
- Public API exposure requires robust validation and resource limits.
- Docker and GPU behavior must be tested on the actual target host.

Error-analysis workflow

```
1. Collect incorrect predictions from evaluation.
2. Inspect true label, predicted label, and scores.
3. Check for blur, cropping, atypical images, or label issues.
4. Review per-class support and confusion patterns.
5. Form one hypothesis at a time.
6. Experiment using training/validation data.
7. Keep test-set use limited and report it honestly.
```

Possible next improvements

Idea	Potential benefit	Caution
Systematic augmentation	Robustness to plausible variation	Strong transforms may remove breed cues.
LR scheduler / early stopping	Training stability	Tune using validation data.
Class-aware sampling or loss	May help weak classes	Can affect calibration or other classes.
Higher resolution	More fine-grained details	More VRAM and latency.
Alternative backbone	Accuracy/speed tradeoff	Requires fair benchmark.
Confidence calibration	More meaningful confidence	Needs held-out calibration data.
Unknown-class rejection	Avoid always forcing a known breed	Needs a defined method and evaluation.
Automated API tests	Catch regressions	Tests must match API contract.

Do not tune repeatedly on the test set

Repeatedly changing the model after inspecting test errors makes the test set part of development and can produce optimistic results. Use training and validation data for experiments, then reserve a clean test set for final evaluation.

15. Interview-ready project explanation

60-second answer

“I built an end-to-end pet-breed classification pipeline using the Oxford-IIIT Pet dataset with 37 cat and dog breeds. I used ImageNet-pretrained ResNet-18, replaced its classifier with a 37-output layer, froze most of the backbone, and fine-tuned layer4 and the classifier. I selected the checkpoint using validation performance and evaluated it on a separate test set of 1,112 images. Best validation accuracy was 90.33%; held-out test accuracy was 89.57%, with 996 correct predictions. I exposed inference through a FastAPI endpoint that accepts an image and returns a breed prediction. The project connects model training and evaluation with inference and API engineering.”

Two-minute answer

“The project uses Oxford-IIIT Pet with 37 breed classes and separate training, validation, and test splits of 5,141, 1,096, and 1,112 images. Images are resized to 224 × 224 and normalized consistently with the pretrained ResNet-18 weights. I started from ImageNet weights because they encode useful visual features, replaced the original classifier with a 37-class linear layer, froze most of the backbone, and fine-tuned layer4 plus the classifier. I used CrossEntropyLoss for multiclass classification and selected the checkpoint with the best validation performance. Best validation accuracy was 90.33%. On the held-out test set, 996 of 1,112 images were correct, giving 89.57% accuracy and 0.3728 test loss. I examined macro and weighted F1 and per-class performance, which showed some visually similar breeds were harder. I separated inference from training and exposed prediction through FastAPI. Important deployment concerns include consistent preprocessing and label mapping, upload validation, loading the model once, documenting dependencies, and testing the actual deployed runtime.”

Report results precisely

- Say “89.57% accuracy on this held-out test set,” not that every real-world prediction is 89.57% certain.
- Give the test count and correct predictions: 996 / 1,112.
- Do not claim a CPU-to-GPU speedup unless it was measured under comparable conditions.
- Do not claim Docker deployment is complete unless the container was built, run, and tested.
- Describe softmax as probability-like scores unless calibration has been measured.
- Name one limitation and one measurable next experiment.

Project story structure

Part	What to explain
Problem	Classify one image into one of 37 known breeds.
Method	Pretrained ResNet-18, replaced head, partial fine-tuning.
Evidence	90.33% validation; 89.57% test on 1,112 examples.
Engineering	Inference, FastAPI contract, validation, testing, packaging.

16. Interview questions with model answers

Try answering each aloud before reading the answer. Expand from these concise responses by pointing to the relevant code in your repository.

1. What is transfer learning?

Reusing representations learned on another dataset/task and adapting them. This project starts from ImageNet-pretrained ResNet-18.

2. Why ResNet-18?

It is a relatively compact residual CNN with available pretrained weights and a practical accuracy/compute balance.

3. What does a skip connection do?

It adds an identity or projected shortcut to a learned residual transformation, helping information and gradients flow.

4. Why replace the final layer?

ImageNet's original head has the wrong label set; this task requires 37 outputs.

5. Feature extraction versus fine-tuning?

Feature extraction freezes the backbone and trains a head. Partial fine-tuning also updates selected pretrained layers.

6. Why freeze most layers?

Preserve useful features, reduce trainable parameters and computation, and reduce the risk of damaging pretrained representations.

7. Why unfreeze layer4?

It allows high-level features to adapt to pet-breed differences while earlier features stay mostly fixed.

8. Why different learning rates?

The new head may need larger updates; pretrained features usually need smaller changes.

9. What are logits?

Raw class scores before softmax; they are not normalized probabilities.

10. Why pass logits to CrossEntropyLoss?

PyTorch performs log-softmax and negative log-likelihood internally in a numerically stable way.

11. What is backpropagation?

Chain-rule computation of loss gradients with respect to parameters through the computational graph.

12. What does the optimizer do?

Uses gradients to update trainable parameters; Adam-family optimizers adapt updates using gradient statistics.

13. What is learning rate?

The scale of parameter updates. Too high can destabilize learning; too low can slow it.

14. Training, validation, and test?

Training updates weights; validation guides development; test estimates final performance after decisions are complete.

15. Why save the best checkpoint?

The final epoch may be worse than an earlier epoch on validation data.

16. What is overfitting?

Learning training-specific details that do not generalize, often visible as validation performance stalls while training improves.

17. Explain the result.

996 / 1,112 test images were correct, so accuracy was approximately 89.57% on this held-out set.

18. Why macro F1?

It weights classes equally and can expose poor performance on smaller or difficult breeds.

19. Why can softmax confidence mislead?

It normalizes scores but does not guarantee calibration; models can be overconfident on wrong or unfamiliar inputs.

20. What does model.eval() do?

Switches BatchNorm and Dropout to evaluation behavior; it does not itself disable gradient tracking.

21. Why inference_mode?

Inference does not need gradients, so avoiding autograd reduces overhead and memory use.

22. Why match preprocessing?

Different resize, normalization, channel order, or class mapping can shift inputs or mislabel outputs.

23. What does FastAPI do here?

Provides an HTTP interface so a client uploads an image and receives a prediction.

24. What does POST /predict mean?

The client submits data for processing, commonly an image as multipart/form-data.

25. What does HTTP 415 mean?

The server rejected the request media type, such as incompatible content type for a file-upload route.

26. What is input validation?

Checking presence, type, size, and decodability before preprocessing and inference.

27. What is Docker?

A way to package an app and runtime dependencies into an image that runs as a container.

28. Image versus container?

An image is the package/template; a container is a running instance.

29. Does Docker automatically use the host GPU?

No; compatible drivers, runtime/toolkit, and framework dependencies are required.

30. How would you improve the project?

Inspect per-class errors, test one hypothesis at a time on development data, and preserve a fair held-out test protocol.

31. What is data leakage?

Information from test/validation or duplicate examples improperly influences training or model selection.

32. What would you monitor in production?

Latency, errors, upload size, memory, model-load health, request volume, and shifts in inputs/predictions.

17. Quick revision sheet and glossary

The whole project in one view

```
DATA: Oxford-IIIT Pet, 37 classes
SPLIT: 5,141 train / 1,096 validation / 1,112 test
INPUT: RGB, 224 x 224, compatible normalization
MODEL: ImageNet-pretrained ResNet-18
HEAD: new Linear layer with 37 outputs
FINE-TUNING: most layers frozen; layer4 + classifier trainable
LOSS: CrossEntropyLoss on raw logits
OPTIMIZATION: gradients + optimizer + learning rates
SELECTION: best checkpoint by validation performance
VALIDATION: best accuracy 90.33%
TEST: 996 / 1,112 = 89.57%; loss 0.3728
SERVING: FastAPI root route + image prediction endpoint
DOCKER: must be built and tested separately
```

Core formulas

Concept	Formula / rule
Accuracy	correct / total
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1	2 × precision × recall / (precision + recall)
Residual block (simplified)	y = F(x) + x
Image batch shape	N × C × H × W
Normalization	(channel value – mean) / standard deviation

Glossary

Term	Short definition
Activation	Output of a layer or activation function.
Backbone	Main feature-extracting part of the model.
BatchNorm	Layer that normalizes activations using batch/running statistics.
Checkpoint	Saved model state and optional metadata.
Class mapping	Mapping from class index to breed name.
DataLoader	Utility that batches and iterates over a Dataset.
Fine-tuning	Updating some pretrained parameters on the target task.
Gradient	Derivative of the objective with respect to a parameter.
Inference	Using trained weights to produce predictions.
Normalization	Scaling input channels using specified statistics.
Residual connection	Shortcut added to a learned transformation.
state_dict	Mapping of parameter/buffer names to tensors.
Throughput	Requests processed per unit time.
VRAM	Dedicated GPU memory.

Final self-check

- I can explain why ResNet-18 was chosen instead of training only a custom CNN from scratch.
- I can explain frozen layers, layer4, and the 37-class head.
- I can describe logits, CrossEntropyLoss, backpropagation, optimizer, and learning rate.

- I can explain checkpoint selection and the train/validation/test distinction.
- I can calculate 89.57% from 996 / 1,112 and interpret precision, recall, and F1.
- I can explain inference preprocessing, eval mode, and no-gradient inference.
- I can explain the API contract, upload validation, and HTTP 415.
- I can distinguish Dockerfile, image, container, port mapping, and GPU requirements.
- I can state limitations honestly and propose a measurable next experiment.

End of Stage 2 theory guide. Revise it alongside the code: theory becomes interview-ready when you can explain why each implementation choice exists.